

Implementing block-sparse matrix multiplication kernels using Triton

Priya Mishra¹ Trevor Gale¹ Matei Zaharia¹ Cliff Young² Deepak Narayanan³

Abstract

MegaBlocks is the state-of-the-art system for efficient training of MoE models based on block-sparse matrix multiplication kernels. The library is currently restricted to a specific block size in the sparse matrices, data type, and GPU architecture. This is due to the CUDA kernels used for the block-sparse matrix products in the MoE layers. These kernels have been hand-tuned and manually optimized to obtain the highest performance for a specific choice of parameters.

In this work, we evaluate re-writing these kernels in Triton, a Python-embedded domain specific language (DSL) for high-performance kernels for GPUs. We show that it is possible to achieve same levels of performance as the hand-tuned CUDA kernels, while maintaining portability across GPU architectures and easily supporting different block sizes and data types without any code changes. We identify the challenges and advantages of using Triton in implementing these block-sparse matrix multiplication kernels.

1. Introduction

MegaBlocks is the state-of-the-art system for efficient training of MoE models on GPUs (Gale et al., 2022). It is based on a reformulation of MoE computations into block-sparse operations that accommodates imbalanced assignment of tokens to experts without requiring dropping tokens or adding padding. Despite its efficiency, MegaBlocks can only be used with a block size of 128 for sparse matrices, fp16 datatype, and Ampere GPU architecture.

This limitation is a direct result of the block-sparse matrix multiplication kernels used by MegaBlocks. These block-sparse kernels are implemented in CUDA and are challenging to generalize across possible block sizes, data types, and

GPU architectures. The choice of these parameters affects design decisions of the kernels, for instance, the shared memory allocation and layout. The kernels have been manually optimized for specific values of these parameters. To maintain the same level of performance for a different set of parameters requires non-trivial implementation changes to the kernels, which makes experimenting across a range of settings challenging.

In this work, we evaluate re-writing these kernels in Triton (Tillet et al., 2019). Triton is a python embedded domain specific language (DSL) for implementing high-performing GPU compute kernels. After some evaluation, we find that our Triton kernels achieve the same level of performance as the highly optimized existing CUDA kernels. The kernels are compact, with $10\times$ fewer lines of code than the CUDA kernels, and are portable across GPU architectures. They support any data type or block size out-of-the-box without requiring any code change while maintaining high performance. Using Triton greatly simplifies the block-sparse kernels, however, we encountered significant road blocks in realizing this. First, Triton is under active development and provides limited documentation which made it challenging to implement and debug performance issues. Second, triggering optimizations like software pipelining required careful experimentation and reformulating the kernel.

After some investigation, we were able to replace the existing CUDA kernels with these Triton kernels. This allows us to generalize MegaBlocks for training MoEs with any configuration while maintaining the training efficiency and model quality¹.

2. Implementation

To describe the matrix products, we use the notation introduced in (Tillet et al., 2019). Each operation is described using three letters. The letters denote the output, left hand input and right hand input respectively. Hence, SDD denotes the operation $sparse = dense \times dense$. The letters following this denote whether the input matrices are transposed. We use N if the matrix is not transposed, and T if the matrix is transposed. For example, TN denotes that the left hand input matrix is transposed and the right hand matrix is

¹Complete code at <https://github.com/stanford-futuredata/stk>

¹Stanford University ²Google Brain ³Microsoft Research. Correspondence to: Priya Mishra <priyamis@cs.stanford.edu>.

Work presented at the ES-FoMo Workshop at 40th International Conference on Machine Learning, Honolulu, Hawaii, USA. PMLR 202, 2023. Copyright 2023 by the author(s).

使用Triton实现块稀疏矩阵乘法内核

普丽娅·米什拉¹ 特雷弗·盖尔¹ 马泰·扎哈里亚¹ 克利夫·杨² 迪帕克·纳拉亚南³

摘要

MegaBlocks是基于块稀疏矩阵乘法内核的高效MoE模型训练先进系统。该库目前受限于稀疏矩阵中的特定块大小、数据类型和GPU架构。这是由于MoE层中用于块稀疏矩阵乘积的CUDA内核所致。这些内核经过手动调优和优化，旨在为特定参数选择实现最高性能。

在这项工作中，我们评估了使用Triton重写这些核函数的效果。Triton是一种嵌入Python的领域特定语言（DSL），专为GPU高性能核函数设计。我们证明，通过Triton可以实现与手动调优的CUDA核函数相同的性能水平，同时保持跨GPU架构的可移植性，并且无需修改代码即可轻松支持不同的块大小和数据类型。我们指出了在使用Triton实现这些块稀疏矩阵乘法核函数时面临的挑战和优势。

1. 引言

MegaBlocks是用于在GPU上高效训练MoE模型的最先进系统（Gale等人，2022年）。它基于将MoE计算重新表述为块稀疏操作，这种操作能够适应令牌到专家的不平衡分配，无需丢弃令牌或添加填充。尽管效率很高，但MegaBlocks仅能用于稀疏矩阵的块大小为128、fp16数据类型以及Ampere GPU架构。

这一限制是MegaBlocks使用的块稀疏矩阵乘法内核的直接结果。这些块稀疏内核在CUDA中实现，并且难以在不同可能的块大小、数据类型和

¹Stanford University ²Google Brain ³Microsoft Research. Correspondence to: Priya Mishra <priyamis@cs.stanford.edu>.

Work presented at the ES-FoMo Workshop at 40th International Conference on Machine Learning, Honolulu, Hawaii, USA. PMLR 202, 2023. Copyright 2023 by the author(s).

GPU架构。这些参数的选择会影响内核的设计决策，例如共享内存的分配和布局。内核已针对这些参数的特定值进行了手动优化。要在不同参数组合下保持相同性能水平，需要对内核进行非平凡的实现更改，这使得在一系列设置中进行实验具有挑战性。

在这项工作中，我们评估了使用Triton（Tillet等人，2019）重写这些内核。Triton是一种用于实现高性能GPU计算内核的Python嵌入式领域特定语言（DSL）。经过一些评估后，我们发现我们的Triton内核达到了与高度优化的现有CUDA内核相同的性能水平。这些内核结构紧凑，代码行数比CUDA内核少 $10\times$ ，并且可在不同GPU架构间移植。它们开箱即用地支持任何数据类型或块大小，无需任何代码更改，同时保持高性能。使用Triton极大地简化了块稀疏内核的实现，然而，我们在实现过程中遇到了重大障碍。首先，Triton正处于积极开发阶段，文档有限，这使得实现和调试性能问题具有挑战性。其次，触发软件流水线等优化需要仔细的实验和内核重构。

经过一番调查，我们得以用这些Triton内核替换现有的CUDA内核。这使得我们能够将MegaBlocks推广至训练任何配置的MoE，同时保持训练效率和模型质量¹。

2. 实施

为了描述矩阵乘积，我们采用(Tillet et al., 2019)中引入的符号表示法。每个运算用三个字母描述，这些字母分别表示输出、左侧输入和右侧输入。因此，SDD表示运算 $sparse = dense \times dense$ 。随后的字母表示输入矩阵是否转置：若矩阵未转置则用N表示，若矩阵转置则用T表示。例如，TN表示左侧输入矩阵转置而右侧矩阵未转置。

¹Complete code at <https://github.com/stanford-futuredata/stk>

not transposed.

We use the hybrid blocked CSR-COO sparse matrix encoding introduced in MegaBlocks (Gale et al., 2022). This format is based on blocked compressed sparse row (BCSR) primitive making it easy to iterate over the nonzero blocks when one of the input matrices is sparse (DSD or DDS). When the output matrix is sparse (SDD), we need to identify both the row and column of each nonzero block which requires looping through the row offsets since BCSR only stores the column indices of the nonzero blocks. The hybrid blocked CSR-COO encoding additionally stores the row indices for each nonzero block for trivial access in SDD kernels. Since the MoE computations require sparse matrix transposition during forward and backward passes, this encoding also maintains metadata for the transposed matrices without explicitly transposing the nonzero blocks. This allows efficient iteration over the transposed sparse matrix through a layer of indirection.

The implementation of our SDD matrix product launches a 1D grid of thread blocks that run in parallel. The number of thread blocks launched is equal to the number of nonzero blocks in the sparse matrix output. Each thread block uses the blocked-COO metadata to identify the row and column of the nonzero block, and hence the sub-matrices of the dense input operands needed to compute the particular nonzero block. Pseudo-code for SDD kernel is shown in Figure 1. The kernel provides a high level abstraction of the matrix multiplication computation simplifying the code. The lower level details such as shared memory allocation or synchronization are handled by the Triton compiler. Note that the kernel is not specific to any block size or data type, allowing ease of experimentation for a range of configurations.

For DSD and DDS, we launch a 2D grid of thread blocks in the size of number of blocks in each dimension. In contrast to SDD, these operations involve indirect memory access of the dense input operand based on the layout of the nonzero blocks in the sparse input operand. We use the sparse matrix metadata to only load the sub-matrices of the dense input that will contribute to a nonzero block in the sparse matrix.

3. Evaluation

We started with implementing the kernels in Triton 2.0, the latest stable release. However, the throughput of the Triton implementation was 50% worse than the CUDA implementation. This major gap in performance was a result of missing software pipelining when accessing the input matrices through a layer of indirection from the sparse matrix metadata. Support for software pipelining with indirect memory accesses was added during our investigation². Hence, we

²<https://github.com/openai/triton/pull/1291>

```
1 def _sdd_kernel (A, B, C, M, N, K,
2                 BLOCK_M, BLOCK_N, BLOCK_K,
3                 row_indices, column_indices):
4
5     # Identify the row & column of non-zero block
6     pid = tl.program_id()
7     pid_m = tl.load(row_indices + pid)
8     pid_n = tl.load(column_indices + pid)
9
10    # Pointers to the input matrices
11    A = A + pid_m * BLOCK_M
12    B = B + pid_n * BLOCK_N
13
14    # Matrix multiplication
15    acc = tl.zeros(BLOCK_M, BLOCK_N)
16    for k in range(0, K // BLOCK_K):
17        a = tl.load(A)
18        b = tl.load(B)
19        acc += tl.dot(a, b)
20        A += BLOCK_K
21        B += BLOCK_K
22
23    # Store to sparse output matrix
24    C = C + pid * BLOCK_M * BLOCK_N
25    tl.store (C, acc)
```

Figure 1. Pseudo-code of SDD kernel in Triton.

use a nightly build of Triton for all experiments. Since Triton is being actively developed, it has limited documentation and can have unexpected behavior, for instance, the pipelining issue described above. It is also unclear to which cases the Triton compiler optimizations apply, and we experimented with multiple ways of structuring the main loops in our kernel to trigger software pipelining. We relied on the PTX source code to understand the Triton code compilation and debug performance issues. Adding debugging support (Brahmakshatriya & Amarasinghe, 2023) in Triton can help in identifying these bottlenecks more easily.

In the following sections, we evaluate the Triton implementation (MegaBlocks-Triton) against the CUDA implementation (MegaBlocks-CUDA) of the MegaBlocks kernels.

We note that Blocksparse is an existing library built using Triton for block-sparse matrix multiplication kernels³ (Tillet et al., 2019). However, it assumes that the location of nonzero blocks in the matrices remains constant between kernel invocations. These kernels rely on pre-processing the sparse matrix layout to compute lookup tables which adds a significant overhead. This makes Blocksparse unusable for MoE computations since subsequent training iterations and different MoE layers have different sparsity layout and we would incur the cost of the pre-processing at every step repeatedly. Hence, we did not use Blocksparse in our work.

3.1. Throughput

The performance of blocksparse matrix multiplication kernels is crucial to the training speedups obtained by

³<https://github.com/openai/triton/blob/main/python/triton/ops/blocksparse/matmul.py>

未转置。

我们采用了MegaBlocks (Gale等人, 2022年) 中引入的混合分块CSR-COO稀疏矩阵编码格式。该格式基于分块压缩稀疏行 (BCSR) 原语, 当其中一个输入矩阵为稀疏矩阵 (DSD或DDS) 时, 能够便捷地遍历非零块。当输出矩阵为稀疏矩阵 (SDD) 时, 由于BCSR仅存储非零块的列索引, 我们需要同时确定每个非零块的行与列索引, 这要求遍历行偏移量。混合分块CSR-COO编码额外存储了每个非零块的行索引, 以便在SDD内核中直接访问。由于MoE计算在前向和反向传播过程中需要进行稀疏矩阵转置, 该编码还保留了转置矩阵的元数据, 而无需显式转置非零块。这通过一层间接引用实现了对转置稀疏矩阵的高效遍历。

我们SDD矩阵乘积的实现启动了一个一维线程块网格, 这些线程块并行运行。启动的线程块数量等于稀疏矩阵输出中非零块的数量。每个线程块利用分块COO元数据来识别非零块的行和列, 从而确定计算该特定非零块所需的稠密输入操作数的子矩阵。SDD内核的伪代码如图1所示。该内核提供了矩阵乘法计算的高级抽象, 简化了代码。诸如共享内存分配或同步等底层细节则由Triton编译器处理。值得注意的是, 该内核不特定于任何块大小或数据类型, 便于对各种配置进行实验。

对于DSD和DDS操作, 我们启动一个二维线程块网格, 其尺寸由各维度的块数决定。与SDD操作不同, 这些操作涉及基于稀疏输入操作数中非零块的布局对密集输入操作数进行间接内存访问。我们利用稀疏矩阵的元数据, 仅加载那些会对稀疏矩阵中非零块产生贡献的密集输入子矩阵。

3. 评估

我们最初在最新的稳定版本Triton 2.0中实现了内核。然而, Triton实现的吞吐量比CUDA实现差50%。这一显著的性能差距是由于通过稀疏矩阵元数据间接访问输入矩阵时, 缺少软件流水线支持所致。在我们的研究过程中, 增加了对间接内存访问的软件流水线支持²。因此, 我们

²<https://github.com/openai/triton/pull/1291>

```
1 def _sdd_kernel (A, B, C, M, N, K,
2                 BLOCK_M, BLOCK_N, BLOCK_K,
3                 row_indices, column_indices):
4
5     # Identify the row & column of non-zero block
6     pid = tl.program_id()
7     pid_m = tl.load(row_indices + pid)
8     pid_n = tl.load(column_indices + pid)
9
10    # Pointers to the input matrices
11    A = A + pid_m * BLOCK_M
12    B = B + pid_n * BLOCK_N
13
14    # Matrix multiplication
15    acc = tl.zeros(BLOCK_M, BLOCK_N)
16    for k in range(0, K // BLOCK_K):
17        a = tl.load(A)
18        b = tl.load(B)
19        acc += tl.dot(a, b)
20        A += BLOCK_K
21        B += BLOCK_K
22
23    # Store to sparse output matrix
24    C = C + pid * BLOCK_M * BLOCK_N
25    tl.store (C, acc)
```

图1. Triton中SDD内核的伪代码。

在所有实验中使用Triton的夜间构建版本。由于Triton正处于积极开发阶段, 其文档有限且可能出现意外行为, 例如上文所述的流水线问题。同时, Triton编译器的优化适用范围尚不明确, 我们尝试了多种内核主循环结构以触发软件流水线。我们依赖PTX源代码来理解Triton代码的编译过程并调试性能问题。在Triton中添加调试支持 (Brahmakshatriya & Amarasinghe, 2023) 有助于更轻松地识别这些瓶颈。

在接下来的章节中, 我们将评估Triton实现 (MegaBlocks-Triton) 与MegaBlocks内核的CUDA实现 (MegaBlocks-CUDA) 之间的性能对比。

我们注意到Blocksparse是一个使用Triton构建的现有库, 用于块稀疏矩阵乘法内核³(Tillet等人, 2019)。然而, 它假设矩阵中非零块的位置在多次内核调用之间保持不变。这些内核依赖于对稀疏矩阵布局进行预处理以计算查找表, 这会带来显著的开销。这使得Blocksparse无法用于MoE计算, 因为后续的训练迭代和不同的MoE层具有不同的稀疏布局, 我们将在每一步重复承担预处理的成本。因此, 我们在工作中未使用Blocksparse。

3.1. 吞吐量

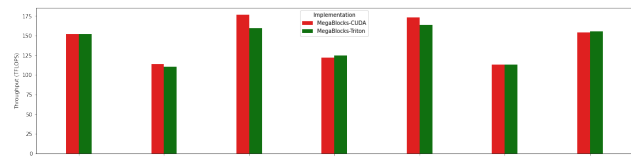
块稀疏矩阵乘法核的性能对于通过

³<https://github.com/openai/triton/blob/main/python/triton/ops/blocksparse/matmul.py>

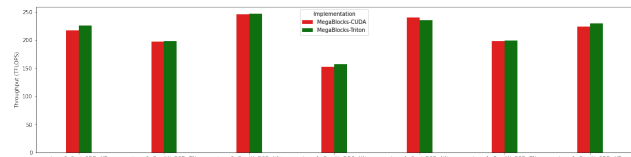
MegaBlocks. We noted in section 2 that kernels using previous versions of Triton had significant performance limitations, necessitating the present CUDA-based kernels used within MegaBlocks.

In this section, we evaluate the throughput obtained with these implementations for various matrix operations and different matrix sizes (sequence length/output matrix side length). The notation used for the matrix operations is described in section 2.

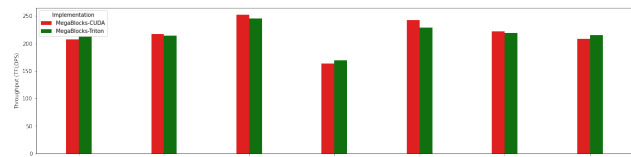
Figure 2 compares the throughput (in TFLOPS) using different implementations across a range of matrix product benchmarks. We observed that MegaBlocks-Triton achieves same levels of throughput as the MegaBlocks-CUDA implementation. The throughput of MegaBlocks-Triton kernels ranges between $0.96\times$ to $1.1\times$ compared to the throughput of MegaBlocks-CUDA kernels and the implementations have the same throughput on average. This makes it feasible to replace the CUDA-based kernels with MegaBlocks-Triton.



(a) Sequence length 8192.



(b) Sequence length 32768.



(c) Sequence length 65536.

Figure 2. Throughput (in TFLOPS) for different matrix product benchmarks using MegaBlocks-Triton and MegaBlocks-CUDA implementations of block-sparse matrix multiplication kernels.

3.2. Source Lines-of-Code

We use source lines-of-code (SLOC) (Wheeler) as a measure to compare the complexity of different implementations. As summarized in Table 1, MegaBlocks-Triton has 10x fewer lines of code than MegaBlocks-CUDA.

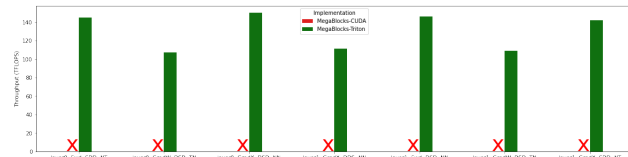
Table 1. Source Lines-of-Code in different implementations

IMPLEMENTATION	SLOC
MEGABLOCKS-TRITON	298
MEGABLOCKS-CUDA	3139

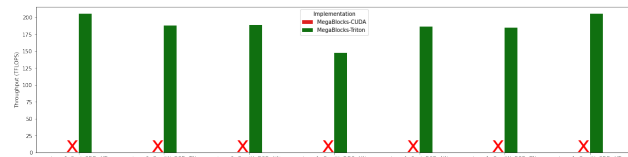
Triton allows us to write the kernels at a high-level abstraction and hides the lower-level optimization details such as shared memory allocation, synchronization, and memory coalescing. This makes the MegaBlocks-Triton kernels compact, portable across different hardware architectures and easily extensible to any block size and data type without requiring any code modification.

3.3. Generality

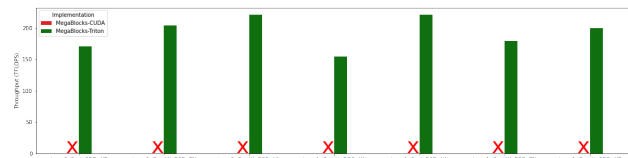
Figures 3 and 4 describe the throughput of MegaBlocks-Triton kernels for bfloat16 datatype and block size 64 respectively. These parameters are not supported by MegaBlocks-CUDA. The average throughput obtained when using bfloat16 across different sequence lengths and matrix operations is 170 TFLOPS, and when using a block size 64 is 130 TFLOPS. MegaBlocks-Triton supports a range of configurations out-of-the-box while maintaining high levels of performance.



(a) Sequence length 8192.



(b) Sequence length 32768.



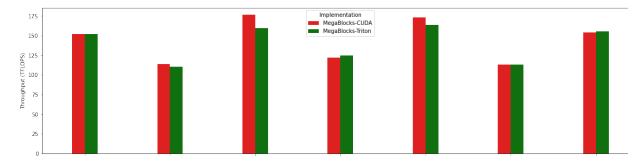
(c) Sequence length 65536.

Figure 3. Throughput (in TFLOPS) for different matrix product benchmarks with block size 128 and bfloat16 datatype.

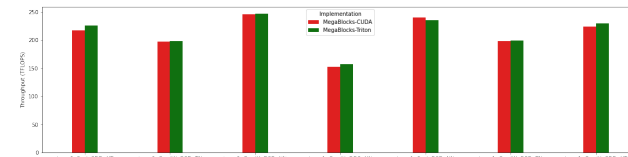
MegaBlocks.我们在第2节中指出，使用早期版本Triton的内核存在显著的性能限制，这促使我们在MegaBlocks内部采用当前基于CUDA的内核。

在本节中，我们评估了这些实现针对不同矩阵运算和不同矩阵尺寸（序列长度/输出矩阵边长）所获得的吞吐量。矩阵运算使用的符号表示方法在第2节中进行了说明。

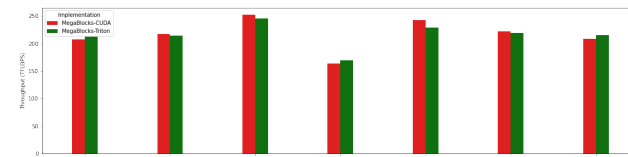
图2比较了在不同矩阵乘积基准测试中使用不同实现的吞吐量（以TFLOPS计）。我们观察到MegaBlocks-Triton实现了与MegaBlocks-CUDA实现相同水平的吞吐量。MegaBlocks-Triton内核的吞吐量范围在 $0.96\times$ 至 $1.1\times$ 之间，与MegaBlocks-CUDA内核的吞吐量相比，这些实现具有相同的平均吞吐量。这使得用MegaBlocks-Triton替换基于CUDA的内核成为可能。



(a) Sequence length 8192.



(b) Sequence length 32768.



(c) Sequence length 65536.

图2. 使用MegaBlocks-Triton和MegaBlocks-CUDA实现的块稀疏矩阵乘法内核在不同矩阵乘积基准测试中的吞吐量（以TFLOPS计）。

3.2. 源代码行数

我们采用源代码行数（SLOC）（Wheeler）作为衡量标准，以比较不同实现的复杂度。如表1所示，MegaBlocks-Triton的代码行数比MegaBlocks-CUDA少10倍。

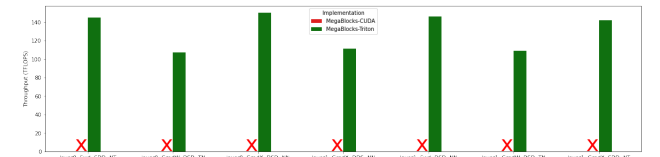
表1. 不同实现中的源代码行数

IMPLEMENTATION	SLOC
MEGABLOCKS-TRITON	298
MEGABLOCKS-CUDA	3139

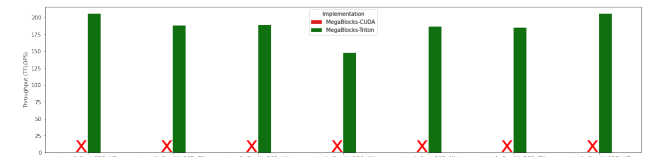
Triton使我们能够以高级抽象层次编写内核，并隐藏了诸如共享内存分配、同步和内存合并等底层优化细节。这使得MegaBlocks-Triton内核变得紧凑，能够跨不同硬件架构移植，并且无需任何代码修改即可轻松扩展到任意块大小和数据类型。

3.3. 通用性

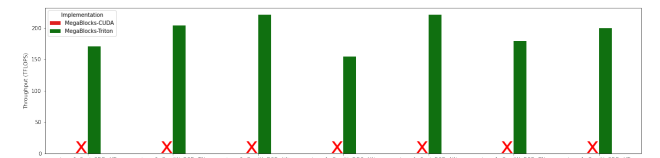
图3和图4分别描述了MegaBlocks-Triton内核在bfloat16数据类型和块大小为64时的吞吐量。这些参数不受MegaBlocks-CUDA支持。使用bfloat16在不同序列长度和矩阵操作下获得的平均吞吐量为170 TFLOPS，而使用块大小64时的平均吞吐量为130 TFLOPS。MegaBlocks-Triton支持多种开箱即用的配置，同时保持高性能水平。



(a) Sequence length 8192.

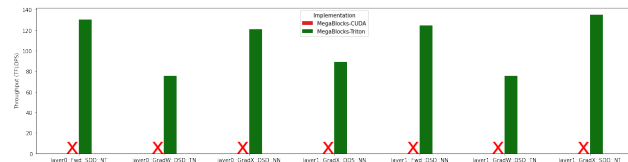


(b) Sequence length 32768.

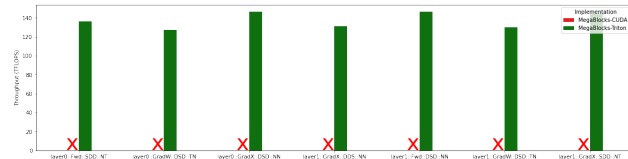


(c) Sequence length 65536.

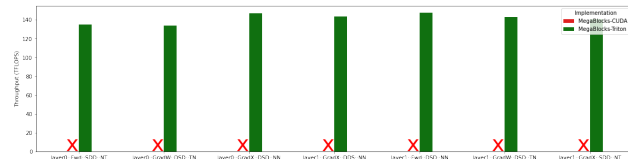
图3. 不同矩阵乘积基准测试的吞吐量（以TFLOPS为单位），块大小为128，数据类型为bfloat16。



(a) Sequence length 8192.



(b) Sequence length 32768.



(c) Sequence length 65536.

Figure 4. Throughput (in TFLOPS) for different matrix product benchmarks with block size 64 and float16 datatype.

4. Conclusion

In this work, we evaluated re-writing the block-sparse matrix multiplication kernels used for MoE computations in MegaBlocks in Triton. Using Triton greatly simplified our kernels, however, it is under active development and has unexpected behavior in some cases. It is challenging to debug since there is limited documentation, and the optimizations applied by the Triton compiler in different kernel formulations can vary significantly. We relied on the PTX source code to debug performance issues and reformulated the kernels multiple times to trigger certain optimizations such as software pipelining.

We show that our Triton kernels (MegaBlocks-Triton) can obtain same performance as the existing CUDA kernels (MegaBlocks-CUDA) making them a feasible replacement. Replacing the existing CUDA kernels with MegaBlocks-Triton removes any restrictions on block size, data type, and architecture. This extends MegaBlocks into a general system for training MoE models across a range of configurations while realizing the same training speedups and model quality.

References

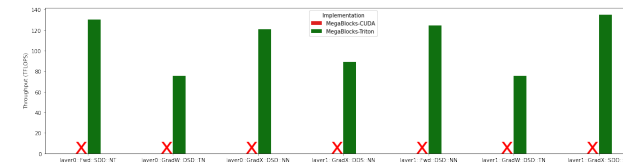
Brahmakshatriya, A. and Amarasinghe, S. D2x: An extensible contextual debugger for modern dsls. In *Pro-*

ceedings of the 21st ACM/IEEE International Symposium on Code Generation and Optimization, CGO 2023, pp. 162–172, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400701016. doi: 10.1145/3579990.3580014. URL <https://doi.org/10.1145/3579990.3580014>.

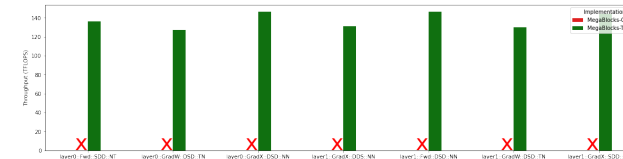
Gale, T., Narayanan, D., Young, C., and Zaharia, M. Megablocks: Efficient sparse training with mixture-of-experts, 2022.

Tillet, P., Kung, H. T., and Cox, D. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, MAPL 2019, pp. 10–19, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450367196. doi: 10.1145/3315508.3329973. URL <https://doi.org/10.1145/3315508.3329973>.

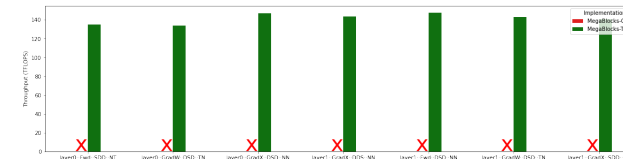
Wheeler, D. A. Sloccount. <https://dwheeler.com/sloccount/>.



(a) Sequence length 8192.



(b) Sequence length 32768.



(c) Sequence length 65536.

图4. 不同矩阵乘积基准测试的吞吐量（以TFLOPS为单位），块大小为64，数据类型为float16。

4. 结论

在这项工作中，我们评估了使用Triton重写MegaBlocks中用于MoE计算的块稀疏矩阵乘法内核。采用Triton极大简化了我们的内核实现，但该框架尚处于积极开发阶段，某些情况下会出现预期外的行为。由于文档有限，且Triton编译器在不同内核实现中应用的优化策略可能存在显著差异，调试工作颇具挑战。我们通过分析PTX源代码来调试性能问题，并多次重构内核以触发软件流水线等特定优化。

我们证明，我们的Triton内核（MegaBlocks-Triton）能够达到与现有CUDA内核（MegaBlocks-CUDA）相同的性能，使其成为一个可行的替代方案。用MegaBlocks-Triton替换现有的CUDA内核，消除了对块大小、数据类型和架构的任何限制。这使MegaBlocks扩展为一个通用系统，能够在多种配置下训练MoE模型，同时实现相同的训练加速和模型质量。

参考文献

Brahmakshatriya, A. 和 Amarasinghe, S. D2x: 一种适用于现代领域特定语言的可扩展上下文调试器。于 *Pro-*

ceedings of the 21st ACM/IEEE International Symposium on Code Generation and Optimization, CGO 2023, 第162–172页，纽约州纽约市，2023年。计算机协会。ISBN 9798400701016. doi: 10.1145/3579990.3580014. URL <https://doi.org/10.1145/3579990.3580014>.

Gale, T., Narayanan, D., Young, C., and Zaharia, M. Megablocks: 专家混合的高效稀疏训练，2022。

Tillet, P., Kung, H. T., 与 Cox, D. Triton: 一种用于分块神经网络计算的中间语言及编译器。收录于 *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, MAPL 2019, 第10–19页，美国纽约州纽约市，2019年。计算机协会。ISBN 9781450367196. doi: 10.1145/3315508.3329973. 网址 <https://doi.org/10.1145/3315508.3329973>。

Wheeler, D. A. 代码行数统计工具。 <https://dwheeler.com/sloccount/>。